ACE Model (Full Data) with Mx

```
library(tinytable)
library(HDInterval)
options(tinytable_html_mathjax = TRUE)
```

```
# Define function for HDI
hdi2 <- function(x, cred_mass = 0.90) {
    hdi_interval <- HDInterval::hdi(x, credMass = cred_mass)
    setNames(hdi_interval, c("hdi_lo", "hdi_hi"))
}
```

In this note, we compare the results of the ACE model with the `umx` package in R, using the same full data as in the other note with Stan.

ACE Model

The classic model assumes that, for a given phenotype, measured by a variable Y^*, it can be decomposed into the additive genetic component A, the common environment component C, and the unique environment component E. For monozygotic (MZ) twins, both A and C are shared. Therefore, for a pair of MZ twins with outcomes Y_1^* and Y_2^*, one can express the model as

$$ Y^{*\textit{MZ}}_1 = A + C + E_1 \\ Y^{*\textit{MZ}}_2 = A + C + E_2 \\ $$

For dizygotic (DZ) twins, with only C completely shared and A shared at a half, the model is

$$ Y^{*\textit{DZ}}_1 = A_1 + C + E_1 \\ Y^{*\textit{DZ}}_2 = A_2 + C + E_2, \\ $$

where A_1 and A_2 have a correlation of 0.5. Note that measurement error is not directly incorporated in the model with a single phenotype, and the presence of random measurement will get absorbed in E.

The heretability h^2 is defined as the proportion of variance explained by A, whereas the commonality c^2 is defined as the proportion of variance explained by C. At the population level, given the twin correlations ρ^{MZ} and ρ^{DZ}, we have the following well-known result:

$$ h^2 = 2(\rho^\textit{MZ} - \rho^\textit{DZ}) \\ c^2 = 2\rho^\textit{DZ} - \rho^\textit{MZ} \\ $$

which is also described in Jöreskog (2020).

Here we do estimation using the `umx` package in R.

```
library(umx)
```

```
Loading required package: OpenMx
```

```
To take full advantage of multiple cores, use:
  mxOption(key='Number of Threads', value=parallel::detectCores()) #now
  Sys.setenv(OMP_NUM_THREADS=parallel::detectCores()) #before library(OpenMx)
```

```
For an overview type '?umx'
```

```

Attaching package: 'umx'
```

```
The following object is masked from 'package:stats':

    loadings
```

```
# Data from Jöreskog (2021)
n_mz <- c(325, 60, 7, 78, 150, 40, 8, 38, 88)
n_dz <- c(202, 97, 19, 96, 112, 64, 46, 56, 66)
```

```
# Convert to long format data
# Create contingency table
tbl_mz <- matrix(n_mz, nrow = 3, ncol = 3)
tbl_dz <- matrix(n_dz, nrow = 3, ncol = 3)

# Convert to long format for MZ
mz_table <- as.data.frame(as.table(tbl_mz))
names(mz_table) <- c("twin1", "twin2", "Freq")
mz_long <- mz_table[rep(seq_len(nrow(mz_table)), mz_table$Freq), ]
mz_long <- mz_long[, c("twin1", "twin2")]
mz_long$twin1 <- as.ordered(mz_long$twin1)
mz_long$twin2 <- as.ordered(mz_long$twin2)
mz_long$zygosity <- "MZ"
rownames(mz_long) <- NULL

# Convert to long format for DZ
dz_table <- as.data.frame(as.table(tbl_dz))
names(dz_table) <- c("twin1", "twin2", "Freq")
dz_long <- dz_table[rep(seq_len(nrow(dz_table)), dz_table$Freq), ]
dz_long <- dz_long[, c("twin1", "twin2")]
dz_long$twin1 <- as.ordered(dz_long$twin1)
```

```r
dz_long$twin2 <- as.ordered(dz_long$twin2)
dz_long$zygosity <- "DZ"
rownames(dz_long) <- NULL

# Combine both datasets
twin_data <- rbind(mz_long, dz_long)
```

```r
# Make the ordinal variables into umxFactors
twin_data[, c("twin1", "twin2")] <- umxFactor(twin_data[, c("twin1", "twin2")])

mz_data <- twin_data[twin_data$zygosity %in% "MZ", ]
dz_data <- twin_data[twin_data$zygosity %in% "DZ", ]

# Model and summary!
m1 <- umxACE(
    selDVs = "twin",
    dzData = dz_data,
    mzData = mz_data,
    sep = "",
    intervals = TRUE,
    tryHard = "ordinal"
)
```

```
Found 1 pair(s) of ordinal variables:'twin1' and 'twin2' (No binary)
```

```
Running ACE with 4 parameters
```

```

Beginning initial fit attempt
```

```
Running ACE with 4 parameters
```

```

 Lowest minimum so far:  5867.41909499216
```

```

Beginning fit attempt 1 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499162
```

```
Beginning fit attempt 2 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 3 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 4 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 5 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499161
```

```
Beginning fit attempt 6 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 7 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499156
```

```
Beginning fit attempt 8 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499151
```

```
Beginning fit attempt 9 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 10 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Retry limit reached
```

```
Solution found
```

```
Final run, for Hessian and/or standard errors and/or confidence intervals
```

```
Running ACE with 4 parameters
```

```
 Solution found!  Final fit=5867.4191 (started at 5907.4077)  (11 attempt(s):
11 valid, 0 errors)
```

```
 Start values from best fit:
```

```
0.00683761070859991,0.845202252899828,0.321951332773407,0.436191539349241
```

```
ACE -2 × log(Likelihood) = 5867.419
```

```


Table: Standardized parameter estimates from a 1-factor Cholesky ACE model. A:
additive genetic; C: common environment; E: unique environment.

|     |    a1|    c1|    e1|
|:----|-----:|-----:|-----:|
|twin | 0.842| 0.321| 0.434|


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
Creating CI-based report!
```

```


|     |a1 |c1 |e1                   |
|:----|:--|:--|:--------------------|
|twin |   |   |0.434 [0.394, 0.478] |


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
?umxPlotACE options: std=T/F, means=T/F, digits=n, strip_zero=T/F, file=, min=, max =
```

```
umxSummary(m1)
```

```
ACE -2 × log(Likelihood) = 5867.42
```

```


Table: Standardized parameter estimates from a 1-factor Cholesky ACE model. A: additive genetic; C: common environment; E: unique environment.

|     |   a1|   c1|   e1|
|:----|----:|----:|----:|
|twin | 0.84| 0.32| 0.43|


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
Creating CI-based report!
```

```


|     |a1 |c1 |e1                |
|:----|:--|:--|:-----------------|
|twin |   |   |0.43 [0.39, 0.48] |


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept |  0.01|  0.01|
```

```
?umxPlotACE options: std=T/F, means=T/F, digits=n, strip_zero=T/F, file=, min=,
max =
```

```
umxConfint(m1, run = TRUE)
```

```
Running ACE with 4 parameters
```

```
               lbound estimate ubound lbound Code ubound Code
top.a_std[1,1]  0.741    0.842     NA           0          NA
top.c_std[1,1]     NA    0.321  0.499          NA           0
top.e_std[1,1]  0.394    0.434  0.478           0           0
```

Using OpenMx

Path specification

```
# Path objects for Multiple Groups
ovs <- c("twin1", "twin2")
lvs <- c("A1", "C1", "E1", "A2", "C2", "E2")
# Common paths
paths <- list(
    # variances of latent variables
    mxPath(from = lvs, arrows = 2, free = FALSE, values = 1),
    # means of latent variables
    mxPath(from = "one", to = lvs, arrows = 1, free = FALSE, values = 0),
    # means of observed variables (constrained to zero)
    mxPath(
        from = "one",
        to = ovs,
        arrows = 1,
        free = FALSE,
        values = 0,
        labels = "mean"
    ),
    # covariance between C1 & C2
    mxPath(from = "C1", to = "C2", arrows = 2, free = FALSE, values = 1),
    # path coefficients for twin 1
    mxPath(
        from = lvs[1:3],
        to = ovs[1],
        arrows = 1,
```

```
        free = TRUE,
        values = .5,
        label = c("a", "c", "e")
    ),
    # path coefficients for twin 2
    mxPath(
        from = lvs[4:6],
        to = ovs[2],
        arrows = 1,
        free = TRUE,
        values = .5,
        label = c("a", "c", "e")
    ),
    # thresholds
    mxThreshold(vars = ovs, nThresh = 2)
)
model_mz <- mxModel(
    "MZ",
    type = "RAM",
    manifestVars = ovs,
    latentVars = lvs,
    mxData(observed = mz_data, type = "raw"),
    paths,
    # covariance between A1 & A2 in MZ twins
    mxPath(from = "A1", to = "A2", arrows = 2, free = FALSE, values = 1)
)
model_dz <- mxModel(
    "DZ",
    type = "RAM",
    manifestVars = ovs,
    latentVars = lvs,
    mxData(observed = dz_data, type = "raw"),
    paths,
    # covariance between A1 & A2 in DZ twins
    mxPath(from = "A1", to = "A2", arrows = 2, free = FALSE, values = 0.5)
)
model_ace <- mxModel(
    "ACE",
    model_mz,
    model_dz,
    mxFitFunctionMultigroup(c("MZ", "DZ"))
)
ace_fit <- mxRun(model_ace)
```

```
Running ACE with 3 parameters
```

```
summary(ace_fit)
```

```
Summary of ACE

free parameters:
  name matrix   row col  Estimate  Std.Error A
1    a   MZ.A twin1  A1 0.7906171 0.05035096
2    c   MZ.A twin1  C1 0.4329848 0.08285485
3    e   MZ.A twin1  E1 0.3900471 0.01903257

Model Statistics:
               |  Parameters  |  Degrees of Freedom  |  Fit (-2lnL units)
       Model:              3                  3101              6110.135
   Saturated:             NA                    NA                    NA
Independence:             NA                    NA                    NA
Number of observations/statistics: 1552/3104

Information Criteria:
      |  df Penalty  |  Parameters Penalty  |  Sample-Size Adjusted
AIC:      -91.86507              6116.135                6116.150
BIC:   -16673.84145              6132.177                6122.647
To get additional fit indices, see help(mxRefModels)
timestamp: 2026-06-29 22:26:58
Wall clock time: 0.05335736 secs
optimizer:  SLSQP
OpenMx version number: 2.22.11
Need help?  See help(mxSummary)
```

```
# additive genetic variance, a^2
A <- mxEval(a * a, ace_fit)
# shared environmental variance, c^2
C <- mxEval(c * c, ace_fit)
# unique environmental variance, e^2
E <- mxEval(e * e, ace_fit)
# total variance
V <- (A + C + E)
# standardized A
a2 <- A / V
# standardized C
```

```
c2 <- C / V
# standardized E
e2 <- E / V
# table of estimates
est_ace <- rbind(cbind(A, C, E), cbind(a2, c2, e2))
# likelihood of ACE model
ll_ace <- mxEval(fitfunction, ace_fit)
```

Matrix specification

```
# ACE Model
#  From  https://static1.squarespace.com/static/58b2481a9f7456906a3b9600/t/5d9c
018be77d8a0f6250b5e0/1570505099516/oneACEvo.pdf
nv <- 1 # number of phenotypes
ovs <- c("twin1", "twin2") # observed variables
nth <- 2 # number of thresholds
# Create Algebra for expected Mean & Threshold Matrices
common_list <- list(
    # Mean matrix
    mxMatrix(
        type = "Zero",
        nrow = 1,
        ncol = 2 * nv,
        name = "zero"
    ),
    # Threshold matrix
    mxMatrix(
        type = "Full",
        nrow = nth,
        ncol = 2 * nv,
        labels = rep(paste0("t", seq_len(nth)), 2),
        values = rep(c(-1, 1), 2),
        free = TRUE,
        name = "thres"
    ),
    # Create Matrices for Variance Components
    mxMatrix(
        type = "Symm",
        nrow = nv,
        ncol = nv,
        free = TRUE,
        values = 0.5,
        label = "VA11",
```

```
        name = "VA"
    ),
    mxMatrix(
        type = "Symm",
        nrow = nv,
        ncol = nv,
        free = TRUE,
        values = 0.5,
        label = "VC11",
        name = "VC"
    )
)
# Create Algebra for expected Variance/Covariance Matrices in MZ & DZ twins
covMZ <- mxAlgebra(expression = VA + VC, name = "cMZ")
covDZ <- mxAlgebra(expression = 0.5 %x% VA + VC, name = "cDZ")
expCovMZ <- mxAlgebra(
    expression = rbind(cbind(1, cMZ), cbind(t(cMZ), 1)),
    name = "expCovMZ"
)
expCovDZ <- mxAlgebra(
    expression = rbind(cbind(1, cDZ), cbind(t(cDZ), V)),
    name = "expCovDZ"
)
# Create Data Objects for Multiple Groups
data_mz <- mxData(observed = mz_data, type = "raw")
data_dz <- mxData(observed = dz_data, type = "raw")
# Create Expectation Objects for Multiple Groups
expMZ <- mxExpectationNormal(
    covariance = "expCovMZ",
    means = "zero",
    dimnames = ovs,
    thresholds = "thres"
)
expDZ <- mxExpectationNormal(
    covariance = "expCovDZ",
    means = "zero",
    dimnames = ovs,
    thresholds = "thres"
)
funML <- mxFitFunctionML()
# Create Model Objects for Multiple Groups
model_mz <- mxModel(
    data_mz,
```

```
    common_list,
    covMZ,
    expCovMZ,
    expMZ,
    funML,
    name = "MZ"
)
model_dz <- mxModel(
    data_dz,
    common_list,
    covDZ,
    expCovDZ,
    expDZ,
    funML,
    name = "DZ"
)
multi <- mxFitFunctionMultigroup(c("MZ", "DZ"))
# Build Model with Confidence Intervals
model_ace <- mxModel(
    "ACEOrdinal",
    model_mz,
    model_dz,
    mxFitFunctionMultigroup(c("MZ", "DZ")),
    mxCI(c("MZ.VA", "MZ.VC"))
)
ace_fit <- mxRun(model_ace)
```

```
Running ACEOrdinal with 4 parameters
```

```
Warning    in    runHelper(model,    frontendStart,    intervals,    silent,
suppressWarnings, : Polite note: Model finished with a larger ordinal error than
we typically expect.
 This may be fine, but you may wish to re-run the model using
 `mxTryHardOrdinal()` in place of `mxRun()` to try for a better fit.
 Expert version: model$output[['maxRelativeOrdinalError']] is
 larger than the mvnRelEps value of  0.005 .
 If this is expected for your model, you might wish to increase `mvnRelEps`, e.g:
 mxOption(NULL, 'mvnRelEps', value= mxOption(NULL, 'mvnRelEps')*5)
 see `?mxOptions`
```

```
Warning: In model 'ACEOrdinal' Optimizer returned a non-zero status code 10.
Starting values are not feasible. Consider mxTryHard()
```

```
ace_fit <- mxTryHardOrdinal(ace_fit, intervals = TRUE)
```

```
Running ACEOrdinal with 4 parameters
```

```

Beginning initial fit attempt
```

```
Running ACEOrdinal with 4 parameters
```

```
 OpenMx status code 10 not in list of acceptable status codes, (0,1,5,6)
```

```

Beginning fit attempt 1 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```

 Lowest minimum so far:  5868.07899958148
```

```

Beginning fit attempt 2 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```

 Lowest minimum so far:  5868.07899958142
```

```

Beginning fit attempt 3 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 4 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 5 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
 Lowest minimum so far:  5868.07899958141
```

```
Beginning fit attempt 6 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
 Lowest minimum so far:  5868.07899958141
```

```
Beginning fit attempt 7 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 8 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 9 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 10 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Retry limit reached
```

```
Solution found
```

```
Final run, for Hessian and/or standard errors and/or confidence intervals
```

```
Running ACEOrdinal with 4 parameters
```

```
 Solution found!  Final fit=5868.079 (started at 7872.1302)  (11 attempt(s): 11
valid, 0 errors)
```

```
 Start values from best fit:
```

```
-0.0774833090316815,0.86638558137779,0.72949598584114,0.0828334774137957
```

```
summary(ace_fit)
```

```
Summary of ACEOrdinal

free parameters:
  name   matrix row   col    Estimate  Std.Error A
1   t1 MZ.thres   1 twin1 -0.07748331 0.02681506
2   t2 MZ.thres   2 twin1  0.86638558 0.03046225
3 VA11    MZ.VA   1     1  0.72949599 0.08171329
4 VC11    MZ.VC   1     1  0.08283348 0.07616335

confidence intervals:
          lbound   estimate    ubound note
VA11  0.57342306 0.72949599 0.8942383
```

```
VC11 -0.07297661 0.08283348 0.2260822

Model Statistics:
               |  Parameters  |  Degrees of Freedom  |  Fit (-2lnL units)
       Model:              4                  3100              5868.079
   Saturated:             NA                    NA                    NA
Independence:             NA                    NA                    NA
Number of observations/statistics: 1552/3104

Information Criteria:
      |  df Penalty  |  Parameters Penalty  |  Sample-Size Adjusted
AIC:       -331.921              5876.079                5876.105
BIC:     -16908.550              5897.468                5884.761
To get additional fit indices, see help(mxRefModels)
timestamp: 2026-06-29 22:26:59
Wall clock time: 0.12328 secs
optimizer:  SLSQP
OpenMx version number: 2.22.11
Need help?  See help(mxSummary)
```

Comparison with Stan results

```
stan_ace_fit <- readRDS(here::here("notebooks", "ace_fit_full.rds"))
stan_ace_s_fit <- readRDS(here::here("notebooks", "ace_s_fit_full.rds"))
stan_ace_res <- stan_ace_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)
stan_ace_s_res <- stan_ace_s_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)
umx_ci <- umxCI(
    m1,
    which = c("a_r1c1", "c_r1c1", "e_r1c1"),
    run = "yes",
```

```
    interval = 0.90
)
```

```
Running ACE with 4 parameters
```

```
               lbound estimate ubound lbound Code ubound Code
top.a_std[1,1]  0.741    0.842     NA           0          NA
top.c_std[1,1]     NA    0.321  0.499          NA           0
top.e_std[1,1]  0.394    0.434  0.478           0           0
a_r1c1          0.755    0.845  0.935           0           0
c_r1c1             NA    0.322  0.482          NA           0
e_r1c1          0.402    0.436  0.473           3           3
[1] "NA: "
[1] "3: The nonlinear constraints and bounds could not be satisfied. The problem
may have no feasible solution."
```

```
Table: Computed CIs in model ACE
```

```
umx_res <- umx_ci$output$confidenceIntervals[
    c("a_r1c1", "c_r1c1", "e_r1c1"),
    c("estimate", "lbound", "ubound")
]^2
```

```
drop_leading_zero <- function(x) sub("^(-?)0\\.", "\\1.", x)

fmt_num <- function(x) drop_leading_zero(sprintf("%.3f", x))

fmt_ci <- function(lo, hi) {
    lo_str <- if (is.na(lo)) {
        "\u2014"
    } else {
        drop_leading_zero(sprintf("%.3f", lo))
    }
    sprintf("[%s, %s]", lo_str, drop_leading_zero(sprintf("%.3f", hi)))
}

h2_rho <- stan_ace_res[stan_ace_res$variable == "h2", ]
c2_rho <- stan_ace_res[stan_ace_res$variable == "c2", ]
e2_rho <- stan_ace_res[stan_ace_res$variable == "e2", ]
h2_sim <- stan_ace_s_res[stan_ace_s_res$variable == "h2", ]
```

```r
c2_sim <- stan_ace_s_res[stan_ace_s_res$variable == "c2", ]
e2_sim <- stan_ace_s_res[stan_ace_s_res$variable == "e2", ]

comp_tbl <- data.frame(
    Model = rep(c("Mx", "Stan (unconstrained)", "Stan (simplex)"), 3),
    Estimate = c(
        fmt_num(umx_res["a_r1c1", "estimate"]),
        fmt_num(h2_rho$mean),
        fmt_num(h2_sim$median),
        fmt_num(umx_res["c_r1c1", "estimate"]),
        fmt_num(c2_rho$mean),
        fmt_num(c2_sim$median),
        fmt_num(umx_res["e_r1c1", "estimate"]),
        fmt_num(e2_rho$mean),
        fmt_num(e2_sim$median)
    ),
    `90% CI` = c(
        fmt_ci(umx_res["a_r1c1", "lbound"], umx_res["a_r1c1", "ubound"]),
        fmt_ci(h2_rho$q5, h2_rho$q95),
        fmt_ci(h2_sim$hdi_lo, h2_sim$hdi_hi),
        fmt_ci(umx_res["c_r1c1", "lbound"], umx_res["c_r1c1", "ubound"]),
        fmt_ci(c2_rho$q5, c2_rho$q95),
        fmt_ci(c2_sim$hdi_lo, c2_sim$hdi_hi),
        fmt_ci(umx_res["e_r1c1", "lbound"], umx_res["e_r1c1", "ubound"]),
        fmt_ci(e2_rho$q5, e2_rho$q95),
        fmt_ci(e2_sim$hdi_lo, e2_sim$hdi_hi)
    ),
    check.names = FALSE
)

tt(comp_tbl) |>
    group_tt(i = list("$h^2$" = 1, "$c^2$" = 4, "$e^2$" = 7))
```

Model	Estimate	90% CI
h^2		
Mx	.714	[.571, .875]
Stan (unconstrained)	.709	[.573, .846]
Stan (simplex)	.699	[.588, .810]
c^2		
Mx	.104	[—, .233]

Model	Estimate	90% CI
Stan (unconstrained)	.100	[-.030, .224]
Stan (simplex)	.110	[.001, .203]
e^2		
Mx	.190	[.162, .224]
Stan (unconstrained)	.192	[.161, .225]
Stan (simplex)	.191	[.163, .224]